Type Classes via Dictionaries

Season: 7 · Advanced FP · Category: Functional · Level: Advanced

Intent

- Pass Functor/Monad instances explicitly.

Problem

- Without this pattern, code tends toward: New things up with conditionals and duplicated construction.

Real-world Analogy

- TypeScript interfaces without implicits.

When to Use

- Serious FP architecture in Java.
- The design force named in Intent shows up more than once.

When Not to Use

- CRUD apps that don't need effect algebras.
- A simpler language feature (record, sealed switch, function value) already solves it.

Pattern Structure

- See mermaid diagram in explanation.md

Classic Implementation

```
package com.buildwithwaqar.patterns.functional.typeclass;

/** Higher-kinded encoding: Kind&lt;F, A&gt; ≈ F&lt;A&gt;. */
public interface Kind<F, A> {}
```

Modern Java 25 / Functional

- Witness types emulate higher-kinded types.

Worked Example

```
See the JUnit test beside the implementation.
Run: mvn -Dtest=*Test test
Focus package matching: functional/typeclass/
```

Pitfalls

- Encodings are educational — production may use libraries.
- Tagless Final needs Kind encoding without HKTs.

Interview Q&A

- Q: When to pick Type Classes via Dictionaries? A: When you need testable effect boundaries or pure state threading.
- Q: How would you test this? A: Prefer injecting interfaces or interpreters; keep demos deterministic (fixed clocks, canned inputs).

Key Takeaways

- Pass Functor/Monad instances explicitly.
- Prefer Java 25 sealed types / records / virtual threads where they simplify the classic shape.
- Ship working tests with every episode.

Video Script Highlights

- [0:00] Hook — Open with the analogy: TypeScript interfaces without implicits. [0:40] Intent — State the design force in one sentence: Pas...